



# Nip the Nits in the Bud

Turning a recurring review comment into a `go/analysis` linter.

Garrett Denis · Principal Engineer, [skool](#)



# “One clause per line, please.”

## ✗ WHAT CAME UP FOR REVIEW

```
if u != nil && u.Active && u.Verified && (u.Role == "admin" || ownerID == postID) {
```

## ✓ WHAT THE REVIEWER ASKED FOR

```
if u != nil &&  
    u.Active &&  
    u.Verified &&  
    (u.Role == "admin" ||  
     ownerID == postID) {
```

# Everyone agreed. Everyone forgot.

## Settled long before me

Skool engineers asked for one clause per line years before I joined. Nobody disputed it.

## And still slipping

Not from newcomers. Engineers who had been here for years, on both sides of the same comment.

A rule nobody argued with and everybody drifted from.  
You cannot fix that by saying it again.

# Two questions before you type it again

## Objective?

A machine can decide it. No taste, no context, no “depends.”

## Recurring?

It has come up more than twice, and it will come up again.

Both yes? Then it isn't a review comment. It's a lint rule  
nobody has written yet.

# A syntax rule needs three fields off Pass

```
var Analyzer = &analysis.Analyzer{
    Name: "sklint_logicalchain",
    Doc:  "one clause per line",
    Run:  run,
}

func run(pass *analysis.Pass) (any, error) {
    // walk the tree, report
}
```

`pass.Files`      the parsed ASTs

`pass.Fset`      position to line

`pass.Reportf`    emit a diagnostic

`pass.TypesInfo`   once you need types

`Pass` carries plenty more. A syntax rule needs none of it.

# Where does the left side end?

# Where does the right start?

```
a && b
```

```
BinaryExpr
```

```
├ X:  a    // left  
├ Op: &&    // LAND / LOR  
└ Y:  b    // right
```

```
lhsEnd    := lineOf(b.X.End())  
rhsStart  := lineOf(b.Y.Pos())
```

```
if lhsEnd == rhsStart {  
    // clauses share a line: report  
}
```

Two positions and a comparison.

# Find the nodes you care about

```
for _, file := range pass.Files {
    reported := make(map[int]bool)
    ast.Inspect(file, func(n ast.Node) bool {
        b, ok := n.(*ast.BinaryExpr)
        if !ok {
            return true
        }
        if b.Op != token.LAND && // one clause per line,
            b.Op != token.LOR { // in its own source
            return true
        }
    })
}
```

# Compare two lines, report once

```
lhsEnd := pass.Fset.Position(b.X.End()).Line
rhsStart := pass.Fset.Position(b.Y.Pos()).Line
if lhsEnd != rhsStart {
    return true
}
if reported[rhsStart] {    // nested chains
    return true
}
reported[rhsStart] = true
pass.Reportf(b.OpPos, "%s", msgShared)
return true
})
}
```

That is the whole `run` function. Thirty lines, one dependency you already have.



# The fixtures **are** the test

## INVALID/INVALID.GO

```
if true && true { // want "operands of"
}
```

## VALID/VALID.GO

```
if true &&
    false {           // already split
}
_ = a & bb & c        // bitwise: no flag
```

```
analysistest.Run(
    t,
    analysistest.TestData(),
    logicalchain.Analyzer,
    "valid",
    "invalid",
)
```

The `valid` package does the real work. False positives are what get a linter disabled.

# A fix is a byte range and some bytes

```
// one edit per operator on the line
for _, b := range ops {
    at := b.OpPos + Pos(len(b.Op.String()))
    edits = append(edits, analysis.TextEdit{
        Pos: at,
        End: at, // insert, do not replace
        NewText: []byte("\n"),
    })
}
```

- `End == Pos` inserts. Replacing up to the next clause would eat a comment sitting there.
- A chain nests, so one line can need several edits.
- `gofmt` handles the indentation.

# Put it in the linter they already run

## .CUSTOM-GCL.YML

```
version: v2.11.4
name: sklint
plugins:
  - module: 'go.skool.com/skool-lint'
    path: ./
```

## PLUGIN.GO

```
register.Plugin(
    "sklint_logicalchain",
    newStatic(logicalchain.Analyzer),
)
```

## CONSUMER'S .GOLANGCI.YML

```
linters:
  enable:
    - sklint_logicalchain
  settings:
    custom:
      sklint_logicalchain:
        type: module
        original-url: go.skool.com/skool-lint
```

Its own linter name, so `//nolint` can target this rule alone.

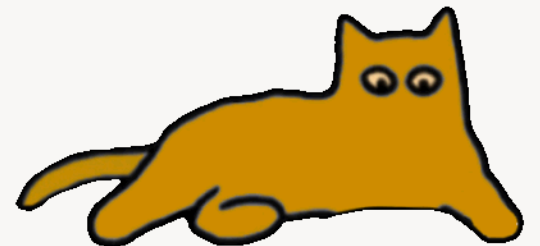
# The comment nobody had to write

```
$ sklint run ./...
```

```
auth.go:10:40: operands of && or || must  
each start on a different line  
(sklint_logicalchain)  
    if u != nil && u.Active && u.Verified && ...  
                                   ^
```

```
1 issues:
```

```
* sklint_logicalchain: 1
```



# The rule matters less than the pattern

- 1 **Objective and recurring?** It belongs in tooling, not a review thread.
- 2 **Ask it of the AST.** This rule fit in one small file; one about types takes more.
- 3 **Run it on your own code first.** Real code exposes what fixtures miss.
- 4 **Ship it pinned and enabled,** so it never surprises anyone mid-sprint.



# Review is for correctness, design, and risk. Nip the rest in the bud.

- A review thread teaches one person, once.
- A failing lint teaches everyone who touches the repo.



We write a lot of Go, and we're hiring.  
[skool.com/careers](https://skool.com/careers)

